Engineering Debug Report

CPU Microbenchmark Suite

Olajide Badejo

July 2026

1 Purpose

This report records the debugging that the clean results in the main report do not show. Each entry states the symptom, the root cause, the fix and why it was chosen over alternatives, and how the fix was verified. The anti optimization fight belongs here, and on this platform it took the form of a clock measurement fight rather than a deleted loop. This document is an equal rank deliverable.

2 Findings

The topology hides the hybrid P and E split

Symptom. The topology reader could not tell performance cores from efficiency cores on the target. The sysfs paths that expose the hybrid split do not exist under WSL2, and every logical CPU reports the same 2 MB L2, so the split cannot be recovered from cache size either.

Root cause. WSL2 runs under a Hyper-V utility virtual machine that flattens the CPU topology it presents to the guest. The hybrid enumeration is a bare metal kernel feature the virtualized sysfs does not carry through.

Fix and why. The reader detects the split when the sysfs files are present and marks it as known; when absent it falls back to the documented Intel enumeration and marks the result as heuristic, so no caller mistakes the guess for a measurement. The real parsing path is still tested against a committed hybrid fixture so the bare metal code cannot rot.

Verification. The topology test passes both a flat WSL style fixture and a hybrid bare metal style fixture, asserting 16 performance and 12 efficiency cores in the hybrid case.

The reported clock does not track turbo

Symptom. The first pointer chase runs reported L1 latency at about 3.2 cycles, which is impossible: the Raptor Cove L1 latency is 5 cycles. The nanosecond timings were right, but the cycle conversion was too low.

Root cause. Cycles were computed from the `cpu MHz` field in `/proc/cpuinfo`, which under WSL2 is pinned at about 3.42 GHz and never moves, even under load, while the pinned core actually boosts to about 5.3 GHz. There is no cpufreq interface to consult either.

Fix and why. A software clock measurement was added: a two billion iteration dependent integer add chain runs at one cycle per iteration, so frequency is iterations over elapsed time. It needs no counter and follows turbo. The wrong cpuinfo value is still recorded for transparency.

Verification. The calibration reports about 5.3 GHz under load, and with it L1 reads 5.0 cycles, L2 16.3, and DRAM 85 to 93 ns, all within the sanity gates. A later single shot instability was

fixed by warming up and taking the maximum over four passes, after a lone 3.81 GHz reading produced an impossible 138 percent FLOPS efficiency.

The plateau detector mixed log and raw error

Symptom. On a synthetic four plateau curve the change point detector returned the wrong boundaries, splitting the flat DRAM plateau instead of the obvious L1 to L2 step.

Root cause. Change point gains were computed in log latency space, the right space, but the child segments after a split had their error recomputed in raw space. A DRAM child then carried a raw error near 65 while every candidate gain was a log space quantity near 2, so a spurious gain of about 65 beat the real L1 to L2 split.

Fix and why. Compute the child segment error in the same log space as the gains. One line.

Verification. The synthetic test recovers the true change points within one index and the plateau means within 25 percent. On the real curve the L1 and L2 boundaries land within one octave of the sysfs capacities.

A main thread barrier race gave impossible bandwidth

Symptom. The STREAM scaling sweep reported thousands of GB/s, physically impossible on a dual channel desktop, and disagreed with the single thread path even at one thread.

Root cause. The parallel region was timed on the main thread across two barriers, and a subtle phase interaction let the timer capture a window in which little work had run, so the minimum over trials latched onto a near zero time.

Fix and why. Move all timing into the workers: each worker times its own chunk between two barriers, and worker zero reduces to the slowest worker per trial and keeps the minimum. The barriers provide the happens before needed to read every worker's time safely, and there is no main thread window to race.

Verification. Bandwidth became physically sane: saturating near 78 GB/s, 87 percent of the theoretical dual channel peak, and the single thread path and the one thread scaling point now agree.

Unsigned underflow poisoned the GEMM test data

Symptom. The GEMM correctness test failed on every element, yet the blocked result matched the reference exactly. The values were infinity.

Root cause. The data was initialized with `(i % 11) - 5` where `i` is unsigned, so residues below 5 underflowed to about 1.8e19 and the products overflowed to infinity. Both kernels agreed at infinity, but the near check computes the absolute difference of two infinities, which is not a number, so every comparison failed.

Fix and why. Cast the residue to a signed integer before subtracting. One line in each of the test and the validator.

Verification. The GEMM test passes for five tile sizes including tiles that do not divide the matrix dimensions.

The BLIS prediction misses the naive kernel, by design

Symptom. The BLIS analytical model predicts a large block K_c near 700, but the naive blocked kernel's best tile uses K_c near 64, a gap of about 3.4 octaves.

Root cause. The model assumes a packed micro kernel whose L1 resident panel of B is only N_r elements wide. The naive kernel does not pack and its inner loop spans the full width of B, so each reused row is the full matrix width and only a few fit in cache, pushing the optimum to a much smaller K_c.

Fix and why. No code fix: this is the reported finding for objective 4. The main report presents both tiles and attributes the gap to the absence of packing. A packed micro kernel is recorded as future work.

Verification. The tile prediction script prints the predicted and empirical tiles and the octave gap, labeled a miss. The roofline shows the same gap from the performance side: the naive GEMM sits below the single core ceiling.

No QEMU floating point quirk, and why that is worth stating

Symptom. The NEON port was expected, from folklore, to show a floating point discrepancy under qemu-user.

Root cause. None appeared. NEON `vfmaq_f32` is a true fused multiply add with a single rounding, matching the x86 FMA, so the same numerical tolerances hold on both.

Fix and why. The width independent FLOPS correctness test and all three NEON STREAM kernel variants pass under qemu-user with the same tolerances as the native build. The absence of a quirk is recorded so the next reader does not go looking for one.

Verification.